

Junior Makers · Class 17

Python Turtle Art Shapes and Patterns

Write real Python code that draws!

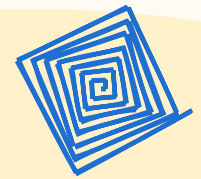
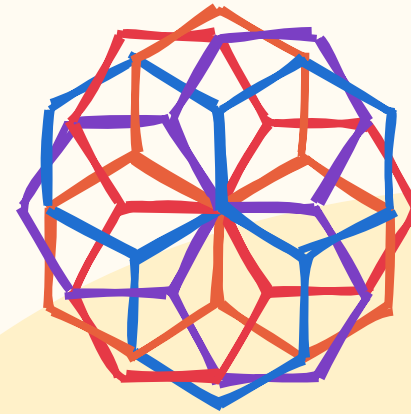
Commands

Loops

Functions

Did you know?

In the 1960s, children programmed a real robot 'turtle' that crawled on paper and drew with a pen. Python's turtle is named after it!





Let's think...

Can you draw this in 10 seconds?



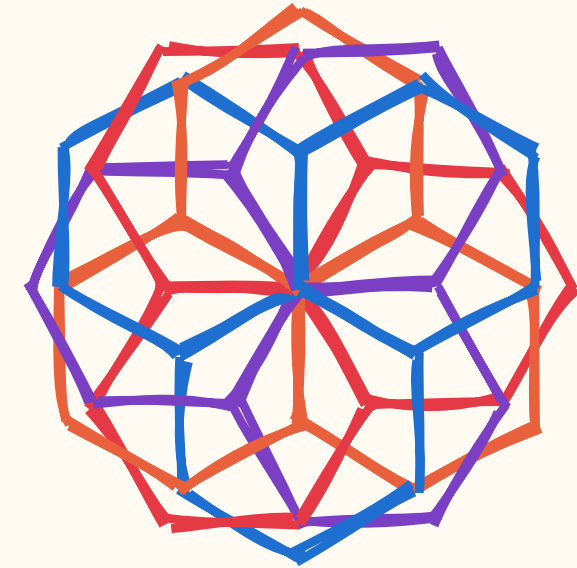
A tricky flower

This flower has **12 hexagons**, each turned a little. Drawing it by hand would take ages and it would never be perfect. A computer can draw it in seconds, perfectly. All we need is to describe the **pattern**.

Use a ruler?

Trace it?

Write code!



Today: you write **Python** code that makes a turtle draw shapes, spirals and your own art.





Learning objectives

By the end of this class I can...



-  **Commands:** move a turtle with **forward**, **left**, **right** and pen commands
Proof: my turtle draws a square without loops
-  **Loops:** use a **for loop** and the **360 ÷ sides** rule to draw any shape
Proof: my loop draws a triangle, square and hexagon
-  **Functions:** make my own **function** and use it to build a pattern
Proof: my `polygon()` function makes a flower or spiral
-  **Debug:** find and fix **bugs** in turtle code
Proof: I fixed one bug and can say what it was





Objective 2 · Loops · video

Video time!



What are Loops? Coding for Kids

Kodable · about 4 minutes · youtu.be/r3Ti5Xp9W8A

Watch for...

- 1 What is a loop?
- 2 Why do programmers use loops instead of writing the same thing again and again?
- 3 Find one loop in your daily life from the video.



After the video: Tell your partner 3 things you repeat every day. How would you write one as a loop?



Class game

Human turtle

How to play

1. One student is the **turtle**, holding a marker on a big sheet of paper on the floor.
2. The class may only say: **FORWARD n steps**, **LEFT 90**, **RIGHT 90**, **PEN UP**, **PEN DOWN**.
3. Draw a square. Then a staircase. Then a triangle (tricky!).
4. Count how many commands each shape needed.

Then discuss

Which commands did you say again and again? How could we say them just once?

Big idea

When you repeat the same commands, a loop can do it for you.





Objective 1 · Commands

Talk to the turtle



The idea

The turtle starts in the **middle** of the screen, facing **right**.

- **t.forward(100)**: walk 100 steps, drawing a line
- **t.left(90)**: turn left 90 degrees
- **t.penup()** / **t.pendown()**: lift or lower the pen
- **t.color("red")**, **t.pensize(5)**: change the pen

Example: giving directions



If you tell a friend 'walk 10 steps, turn left, walk 5 steps', they arrive exactly where you want. The turtle follows your directions just as exactly, but never gets tired.





Objective 1 · Commands · Build step 1 of 4

Draw a square, the long way



```
import turtle
t = turtle.Turtle()
t.shape("turtle")

t.forward(100)
t.left(90)
t.forward(100)
t.left(90)
t.forward(100)
t.left(90)
t.forward(100)
t.left(90)

turtle.done()           # keeps the window open
```

Thonny

Open **Thonny**, type the code, save as **square.py**, press the green **Run** button.

Exactly!

Python is fussy: **t.forward(100)** works, **t.Forward (100)** does not. Check capitals and brackets.

Count

This needed 8 lines of moves. Remember that number!





Objective 2 · Loops

A for loop repeats for you



The idea

`for i in range(4):` means 'do the indented lines 4 times'.

The **indent** (4 spaces) shows which lines are inside the loop. Lines that are not indented run **after** the loop finishes.

Our square goes from 8 lines to 3!

Colon!

Forgot the colon : at the end of the for line? Python shows a `SyntaxError`.

Example: skipping rope

```
for i in range(4):  
    t.forward(100)  
    t.left(90)
```

Your PE teacher says 'skip 20 times'. She doesn't say 'skip, skip, skip...' 20 times. 'Repeat 20 times' is a loop. Python's version is `for i in range(20):`.



Objective 2 · Loops

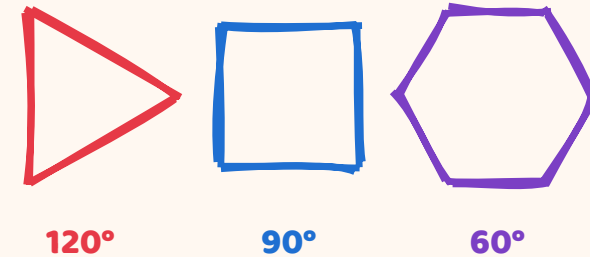
The $360 \div$ sides rule

The idea

To get back to the start, the turtle must turn 360° in total. So each turn is $360 \div \text{number of sides}$:

- triangle: $360 \div 3 = 120^\circ$
- square: $360 \div 4 = 90^\circ$
- hexagon: $360 \div 6 = 60^\circ$
- 36 sides: 10° ... it looks like a **circle**!

Example: walking round the school field



Walk all the way round the field and you end up facing the same way you started. However many corners you turned, they add up to one full turn: 360° .



Objective 2 · Loops · Build step 2 of 4

One loop, any shape



```
import turtle
t = turtle.Turtle()
t.speed(0)                # fastest!

sides = 6                  # try 3, 4, 5, 8, 36
for i in range(sides):
    t.forward(80)
    t.left(360 / sides)

turtle.done()
```

Variable

sides is a variable: a box with a name that holds a number. Change it, and the whole shape changes.

Try

Make sides = 36 and forward(10). Does it look like a circle?



Challenge

What happens with sides = 5 and turning 144 instead? (a star!)





Objective 3 · Functions

Make your own command: a function



The idea

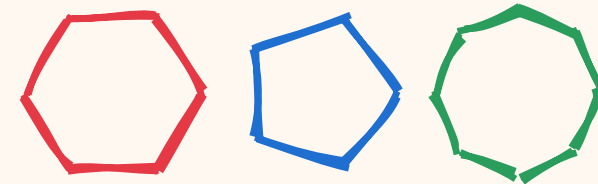
A **function** is a recipe with a name. Write it once, use it anywhere:

```
def polygon(sides, size, colour):
```

Then `polygon(6, 50, "red")` draws a red hexagon.

The values in brackets are **parameters**: they change what the function does each time.

Example: a cake recipe



`polygon(sides, size, colour)`

A recipe called 'make cake' works for chocolate or vanilla, small or big. You just say which flavour and size. A function works the same way with sides, size and colour.





Objective 3 · Functions · Build step 3 of 4

Function + loop = pattern



```
import turtle
t = turtle.Turtle()
t.speed(0)
turtle.bgcolor("black")

def polygon(sides, size, colour):
    t.color(colour)
    for i in range(sides):
        t.forward(size)
        t.left(360 / sides)

colours = ["red", "orange", "yellow", "cyan", "violet"]
for i in range(36):
    polygon(6, 60, colours[i % 5])
    t.left(10)          # 36 × 10 = 360

turtle.done()
```

Loop in a loop

The big loop runs 36 times. Each time, the function runs its own small loop. That is a **nested loop**.

i % 5

Gives 0, 1, 2, 3, 4, 0, 1... so the colours repeat in order.



Check

36 hexagons, each turned 10°, make a full flower.





Objective 3 · Functions · experiment

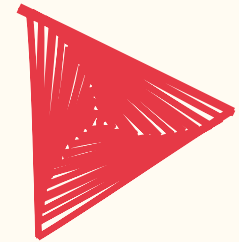
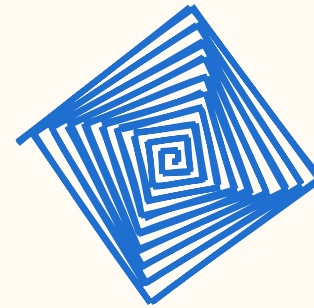
Spirals: change one number



```
import turtle
t = turtle.Turtle()
t.speed(0)
t.pensize(2)

for i in range(120):
    t.forward(i * 2)      # each line a bit longer
    t.left(91)           # try 90, 91, 120, 144

turtle.done()
```

**Predict, then run**

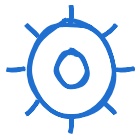
Before you press Run, guess what 91 will look like compared with 90. Then try 120 and 144. Write your guesses down!





Real world

Machines that draw with code



CNC machines

Cut wood and metal by following coordinates



Embroidery machines

Stitch logos from a list of moves



Laser cutters

Burn patterns into wood and card



Games and films

Code draws trees, waves and patterns



Look at a pattern in a carpet or tile. Could you describe it with a loop?

Islamic geometry

Many tile patterns in mosques are made by repeating and rotating one shape, just like our flower.



Test like an engineer

Bug hunt



Bug	Likely cause	Did it happen to you?
Window opens and closes at once	missing turtle.done()	
IndentationError	loop lines not indented by 4 spaces	
Shape doesn't close	turn is not $360 \div \text{sides}$	
NameError: 'Turtle'	capital letters wrong	
Nothing happens for ages	speed too slow → t.speed(0)	



Debug checklist

Read the error message from the bottom up. It tells you the line number!



Every programmer gets bugs. Good programmers are just fast at reading error messages.





Objective 3 · Build step 4 of 4

Make your own artwork



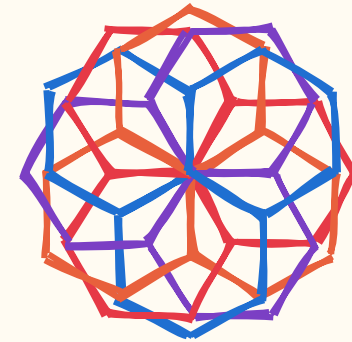
a Plan on paper: which shape, how many times, how much turn?

b Use your **polygon()** function at least twice.

c Use a **loop** to repeat and turn it.

d Add colours with a list, and **turtle.bgcolor()**.

e Give it a title with **t.write("My art", font=("Arial", 20))**.



Check: Your art uses a function, a loop and at least 3 colours.





Objective 4 · Debug · share

Code gallery walk



How it works

Leave your art on screen with your code beside it. Walk round and look at other screens.

At each one, write a sticky note:

★ one thing you **like**

🔧 one number to **change**, and what you think will happen

My art card

Shapes I used

Loop runs ___ times

Turn angle

A bug I fixed





Level up!

Turtle challenges



Choose your challenge



Star: draw a 5-pointed star with a loop (hint: 144°).



Sun: a circle with 12 rays around it.



Random art: use import random to pick random colours and angles.



Keyboard: use `turtle.onkey()` so arrow keys move the turtle.

Think like an engineer

Why does turning 91° make a spiral that twists, but 90° makes a neat square spiral?



Boss level

Draw your name using functions for each letter!





Objective check

Can I really do it? Prove it.



01 Show the commands that draw a square without a loop.



02 What is the turn for a pentagon? Show your loop drawing one.



03 Show your polygon() function. What are its 3 parameters?



04 Tell us one bug you fixed and how you found it.



Rate yourself

Colour one circle each. Red: not yet. Amber: with help. Green: I can explain it to a friend.

Teacher: Print the best artworks with their code and make a 'Code Art' wall.





Show what you know

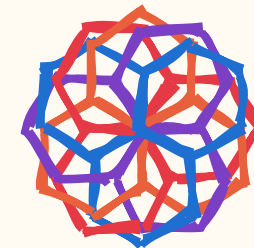
What did we learn?

- ✓ Turtle commands: forward, left, right, penup, pendown
- ✓ `for i in range(n):` repeats the indented lines n times
- ✓ Turn $360 \div \text{sides}$ to draw any regular shape
- ✓ `def` makes a function you can reuse with parameters
- ✓ Change one number, predict, then run



At home

Find a pattern at home (tiles, cloth, a plate). Draw it on paper and write how you'd code it.



Next class:
MakeCode
Arcade: My First
Platform Game!



Exit ticket: what turn angle makes an octagon (8 sides)?

